1 Fondamentaux

- «Déclaration de variable: » `var=1` sera un entier, `var=1.0` (sera un réel), `var="1"` (sera une chaine de caractère), `var=[var1,var2,var3]` (sera une liste), `var=(var1,var2,var3)` sera un tuple.
- «Changer le type d'une variable: » `var=int("1")` (sera un entier), **float** (pour réel), **str**(pour chaine de caractères).
- «Variable saisie au clavier: »
  `var=input("texte s'affichant à l'écran et demandant de saisir une variable")`
  le contenu de la variable saisie au clavier est par défaut une chaine de caractère, il faudra la convertir si on veut des nombres.
- «Opérateurs: » `+` `-` `*` `%` modulo `//` (Quotient de la division euclidienne) `**` (puissance) = (affectation)
- «1e5»Écriture scientifique de 10^5.
- «1j»Nombre complexe i tel que $i^2 = -1$.
- «Opérateurs logiques: » `not` (ou !) `and` (ou &) `or` (ou |) `True` (ou 1) `False` (ou 0)
- «Affichage: » `print(".....",var1, var2,....)`

2 Conditions et boucles

- «Condition si (if)»

```
if (condition1) :
  instruction1
elif (condition2) :
  instruction2
else :
  instruction final
```

- «Boucle pour (for)»

```
for i in range(1,10,1) :
  instruction à répéter pour i allant de 1 à 9  par pas de 1.
```

- «Boucle tant que (while)»

```
while (condition) :
  instruction tant que condition est vraie
```

3 Les listes

Une liste est modifiable.

- «T = [el1 ,el2 , el3]» créer une liste L.
- «list(T)» Transforme un tuple en une liste.
- «range(10)» : liste de 0 à 9 par pas de 1
- «range (2, 20, 3)» liste de 2 à 20 (exclue) par pas de 3
- «Concaténation» L1+L2
- Sélection de liste
  - ➔ «L[1:6:3]» sélection du 2eme au 7eme (exclu) éléments par pas de 3.
  - ➔ «L[-2]» sélection des deux derniers éléments de L.
  - ➔ «L[2:]» sélection de l'élément 2 jusqu'à la fin de L.
  - ➔ «L[::-1]» sélection de L mais inversée.
  - ➔ «L[0][-1]» sélection du premier élément de L et en prendre le dernier élément de celui ci
- «L.reverse()» inverser l'ordre des items d'une liste.
- «var=L.count(élément à compter) » compter combien de fois un élément apparait dans une liste
- «L.append(élément à ajouter)» ajouter un élément à la fin d'une liste.

- «`L.insert(i,x)`» l'élément x dans la liste avant l'élément à la position i (le reste est décalé)
- «`L.pop(i)`» l'élément x dans la liste avant l'élément à la position i (le reste est décalé)
- « `var = len(L)`» longueur d'une liste

4 chaines de caractères

Une chaîne de caractère est non modifiable.

- «`s.upper()`» Mettre tout en majuscule
- «`s.lower()` » Mettre tout en minuscule
- «`s.title()`» Mettre la première lettre de chaque mots en majuscule, le reste en minuscule

5 Tuples :

Un tuple est non modifiable.

- «`T = (el1 ,el2 , el3)`» définit un tuple.
- «`tuple(L)`» transforme la liste L en un tuple.
- «`Permuter des données`» var1, var2 = var2, var1

6 Les fonctions

- «`importer un module`»
  - ➔ `import math`, on sera obligé d'utiliser math. avant d'utiliser un fonction du module `math.sqrt(2)`.
  - ➔ `import numpy as np`, np sera alors un racourci du module numpy `np.exp(1)`.
  - ➔ `from math import sqrt, sin` (on importe seulement la fonctions *sqrt* et *sin* du module *math*)
  - ➔ `from math import *` idem que précédemment mais on importe toutes les fonctions du module.
    Avec **from** il ne faut pas mettre le nom du module, seulement le nom de la fonction, par exemple `sqrt(2)` et non *math.sqrt(2)*.
- «`def`»

```
def  nomDeLaFonction (param1 , param2=valParDefault2 etc... ) :
        """ description de la fonction, Docstring """
        Actions
        return var1, var2
```

  Les variables sont locales SAUF pour les listes qui sont définitivement modifiées !
- «`modules courants`» maths, os (système), random, time, tkinter (fenêtres), numpy, matplotlib (graphique), sympy (calcul formel), httplib (connections http) ...

7 Manipulation de fichier

On importe le module os

- «`fichier = open("monfichier.txt", "r")`» pour ouvrir un fichier en lecture
  Les options sont : **r** lecture seule, **w** écriture, **a** pour ajouter du texte **b** binaire
- «`contents = fichier.read()`» pour la lecture du fichier.
- «`lines = fichier.readlines()`» pour lire une seule ligne.
- «`fichier.close()`» pour fermer le fichier.
- «`fichier.write("Ceci est un texte")`» pour écrire le texte. Si le fichier n'existe pas, il sera créé.
- « `fichier.writeline(line)`» Pour écrire une ligne supplémentaire.

8 Le module random

On importe le module random.

- «seed(n)» pour réinitialiser le générateur de nombres aléatoires en utilisant l'entier n

- «seed()» idem en utilisant l'horloge du système
- «randrange(a,b,h)» pour obtenir un entier aléatoire de $[a;b[$ avec le pas h par défaut $h = 1$
- «randrange(b)» pour obtenir un entier aléatoire de $[0;b[$
- «randint(a,b)» pour obtenir un entier aléatoire de $[a;b]$
- «choice(seq)» pour obtenir un élément au hasard dans la séquence (non vide) seq (string ou liste...)
- «shuffle(seq)» pour "rebattre" aléatoirement la séquence (non vide) seq mais modifie seq
- «sample(seq,k)» pour obtenir k éléments distincts de seq
- «random()» pour obtenir un réel pseudo-aléatoire dans $[0;1[$
- «uniform(a,b)» pour obtenir un réel pseudo-aléatoire dans $[a;b]$

9 Adresses utiles

Voici quelques modules qui pourraient s'avérer utiles... ou même indispensables.
Tout dépend de ce qu'on veut faire avec Python.

- le module os.path (opérations sur les chemins d'accès)
  adresse : http ://docs.python.org/3.3/library/os.path.html

- le module pickle (sauvegardes d'objets Python dans des fichiers)
  adresse : http ://docs.python.org/3.3/library/pickle.html#module-pickle

- le module os (opérations liées au système d'exploitation)
  adresse : http ://docs.python.org/3.3/library/os.html

- le module io (opérations d'entrées-sorties, en mode texte ou binaire)
  adresse : http ://docs.python.org/3.3/library/io.html

- le module turtle (programmer les déplacements d'un crayon d'une tortue Logo à l'écran)
  adresse : http ://docs.python.org/3.3/library/turtle.html

- le module tkinter (conception d'interfaces graphiques, programmation par événements)
  adresse : http ://docs.python.org/3.3/library/tkinter.html

- le module tkinter.ttk (bibliothèque de widgets pour le module tkinter)
  adresse : http ://docs.python.org/3.3/library/tkinter.ttk.html

- le module tkinter.tix (widgets supplémentaires pour le module tkinter)
  adresse : http ://docs.python.org/3.3/library/tkinter.tix.html

- le module sys (paramètres et fonctions spécifiques au système d'exploitation)
  adresse : http ://docs.python.org/3.3/library/sys.html

La liste précédente est très loin d'être exhaustive !